SWIFTRIDE NIGERIA
RELATIONAL SCHEMA

Advanced Database Systems Portfolio

1. Overview

This relational schema translates the SwiftRide Nigeria Entity–Relationship (ER) model into nine relational tables. Primary keys identify records, foreign keys establish relationships, and unique and check constraints support data integrity.

Notation: PK = Primary Key; FK = Foreign Key; UK = Unique Key; NULL = optional; NOT NULL = required.

1.1 PASSENGER

PASSENGER(PassengerID, FirstName, LastName, Email, PhoneNumber, RegistrationDate, AccountStatus)

Stores registered passenger details. Email and PhoneNumber must be unique.

Attribute	SQL Server data type	Key / constraint
PassengerID	INT	PK, NOT NULL
FirstName	VARCHAR(50)	NOT NULL
LastName	VARCHAR(50)	NOT NULL
Email	VARCHAR(100)	UK, NOT NULL
PhoneNumber	VARCHAR(25)	UK, NOT NULL
RegistrationDate	DATETIME2	NOT NULL
AccountStatus	VARCHAR(20)	NOT NULL

1.2 DRIVER

DRIVER(DriverID, FirstName, LastName, Email, PhoneNumber, LicenceNumber, RegistrationDate, DriverStatus)

Stores driver registration information. Email, PhoneNumber and LicenceNumber are unique.

Attribute	SQL Server data type	Key / constraint
DriverID	INT	PK, NOT NULL
FirstName	VARCHAR(50)	NOT NULL
LastName	VARCHAR(50)	NOT NULL
Email	VARCHAR(100)	UK, NOT NULL
PhoneNumber	VARCHAR(25)	UK, NOT NULL
LicenceNumber	VARCHAR(50)	UK, NOT NULL
RegistrationDate	DATETIME2	NOT NULL
DriverStatus	VARCHAR(20)	NOT NULL

1.3 VEHICLE

VEHICLE(VehicleID, RegistrationNumber, Make, Model, ModelYear, Colour, VehicleStatus)

Stores vehicle details. RegistrationNumber must be unique.

Attribute	SQL Server data type	Key / constraint
VehicleID	INT	PK, NOT NULL
RegistrationNumber	VARCHAR(20)	UK, NOT NULL
Make	VARCHAR(50)	NOT NULL
Model	VARCHAR(50)	NOT NULL
ModelYear	SMALLINT	NOT NULL
Colour	VARCHAR(30)	NOT NULL
VehicleStatus	VARCHAR(20)	NOT NULL

1.4 VEHICLE_ASSIGNMENT

VEHICLE_ASSIGNMENT(AssignmentID, DriverID, VehicleID, StartDateTime, EndDateTime, AssignmentStatus)

Maintains driver–vehicle assignment history. Filtered unique indexes can limit each driver and vehicle to one active assignment.

Attribute	SQL Server data type	Key / constraint
AssignmentID	INT	PK, NOT NULL
DriverID	INT	FK → DRIVER.DriverID, NOT NULL
VehicleID	INT	FK → VEHICLE.VehicleID, NOT NULL
StartDateTime	DATETIME2	NOT NULL
EndDateTime	DATETIME2	NULL
AssignmentStatus	VARCHAR(20)	NOT NULL

1.5 FARE_RULE

FARE_RULE(FareRuleID, RuleName, BaseFare, RatePerKilometre, MinimumFare, EffectiveFrom, EffectiveTo, RuleStatus)

Stores current and historical fare rules, including effective dates.

Attribute	SQL Server data type	Key / constraint
FareRuleID	INT	PK, NOT NULL
RuleName	VARCHAR(100)	NOT NULL
BaseFare	DECIMAL(10,2)	NOT NULL
RatePerKilometre	DECIMAL(10,2)	NOT NULL
MinimumFare	DECIMAL(10,2)	NOT NULL
EffectiveFrom	DATETIME2	NOT NULL
EffectiveTo	DATETIME2	NULL
RuleStatus	VARCHAR(20)	NOT NULL

1.6 RIDE

RIDE(RideID, PassengerID, DriverID, FareRuleID, RequestDateTime, PickupLocation, Destination, DistanceKilometres, EstimatedFare, FinalFare, RideStatus, StartDateTime, EndDateTime)

Central operational table. DriverID can be NULL until assignment; final fare and journey timestamps may remain NULL until the ride progresses.

Attribute	SQL Server data type	Key / constraint
RideID	BIGINT	PK, NOT NULL
PassengerID	INT	FK → PASSENGER.PassengerID, NOT NULL
DriverID	INT	FK → DRIVER.DriverID, NULL
FareRuleID	INT	FK → FARE_RULE.FareRuleID, NOT NULL
RequestDateTime	DATETIME2	NOT NULL
PickupLocation	VARCHAR(200)	NOT NULL
Destination	VARCHAR(200)	NOT NULL
DistanceKilometres	DECIMAL(8,2)	NOT NULL
EstimatedFare	DECIMAL(10,2)	NOT NULL
FinalFare	DECIMAL(10,2)	NULL
RideStatus	VARCHAR(20)	NOT NULL

StartDateTime	DATETIME2	NULL
EndDateTime	DATETIME2	NULL

1.7 PAYMENT

PAYMENT(PaymentID, RideID, PaymentDateTime, Amount, PaymentMethod, PaymentStatus, TransactionReference)

Stores payment attempts, including failed and pending transactions. A filtered unique index can enforce at most one successful payment per ride.

Attribute	SQL Server data type	Key / constraint
PaymentID	BIGINT	PK, NOT NULL
RideID	BIGINT	FK → RIDE.RideID, NOT NULL
PaymentDateTime	DATETIME2	NOT NULL
Amount	DECIMAL(10,2)	NOT NULL
PaymentMethod	VARCHAR(20)	NOT NULL
PaymentStatus	VARCHAR(20)	NOT NULL
TransactionReference	VARCHAR(100)	UK, NOT NULL

1.8 RATING

RATING(RatingID, RideID, RatingScore, Comments, RatingDateTime)

Allows zero or one rating per ride because RideID is both a foreign key and unique.

Attribute	SQL Server data type	Key / constraint
RatingID	BIGINT	PK, NOT NULL
RideID	BIGINT	FK → RIDE.RideID, UK, NOT NULL
RatingScore	TINYINT	NOT NULL; CHECK 1–5
Comments	VARCHAR(500)	NULL
RatingDateTime	DATETIME2	NOT NULL

1.9 DRIVER_LOCATION

DRIVER_LOCATION(LocationID, DriverID, Latitude, Longitude, RecordedDateTime)

Stores timestamped driver coordinates for tracking and historical analysis.

Attribute	SQL Server data type	Key / constraint
LocationID	BIGINT	PK, NOT NULL
DriverID	INT	FK → DRIVER.DriverID, NOT NULL
Latitude	DECIMAL(10,7)	CHECK −90 to 90; NOT NULL
Longitude	DECIMAL(10,7)	CHECK −180 to 180; NOT NULL
RecordedDateTime	DATETIME2	NOT NULL

2. Compact Relational Notation

PASSENGER(PassengerID PK, FirstName, LastName, Email UK, PhoneNumber UK, RegistrationDate, AccountStatus)

DRIVER(DriverID PK, FirstName, LastName, Email UK, PhoneNumber UK, LicenceNumber UK, RegistrationDate, DriverStatus)

VEHICLE(VehicleID PK, RegistrationNumber UK, Make, Model, ModelYear, Colour, VehicleStatus)

VEHICLE_ASSIGNMENT(AssignmentID PK, DriverID FK → DRIVER.DriverID, VehicleID FK → VEHICLE.VehicleID, StartDateTime, EndDateTime, AssignmentStatus)

FARE_RULE(FareRuleID PK, RuleName, BaseFare, RatePerKilometre, MinimumFare, EffectiveFrom, EffectiveTo, RuleStatus)

RIDE(RideID PK, PassengerID FK → PASSENGER.PassengerID, DriverID FK → DRIVER.DriverID NULL, FareRuleID FK → FARE_RULE.FareRuleID, RequestDateTime, PickupLocation, Destination, DistanceKilometres, EstimatedFare, FinalFare, RideStatus, StartDateTime, EndDateTime)

PAYMENT(PaymentID PK, RideID FK → RIDE.RideID, PaymentDateTime, Amount, PaymentMethod, PaymentStatus, TransactionReference UK)

RATING(RatingID PK, RideID FK → RIDE.RideID UK, RatingScore, Comments, RatingDateTime)

DRIVER_LOCATION(LocationID PK, DriverID FK → DRIVER.DriverID, Latitude, Longitude, RecordedDateTime)

3. Relationship Cardinalities

Relationship	Cardinality	Explanation
PASSENGER → RIDE	1:M	A passenger may request multiple rides; each ride belongs to one passenger.
DRIVER → RIDE	1:M; optional driver on RIDE	A driver may fulfil many rides; a ride may initially have no driver.
FARE_RULE → RIDE	1:M	One fare rule may be referenced by multiple rides.
DRIVER → VEHICLE_ASSIGNMENT	1:M over time	Assignment records preserve a driver's vehicle history.
VEHICLE → VEHICLE_ASSIGNMENT	1:M over time	Assignment records preserve a vehicle's driver history.
RIDE → PAYMENT	1:M	A ride may have multiple payment attempts.
RIDE → RATING	1:0..1	A ride may have no rating or one rating.
DRIVER → DRIVER_LOCATION	1:M	A driver may generate many location records.

4. Normalisation and Integrity

1NF: attributes are individual fields and repeating groups are avoided. 2NF: tables use single-column primary keys, and non-key attributes depend on their row's key. 3NF: passenger, driver, vehicle and fare-rule details are maintained in separate tables and referenced through foreign keys, reducing duplication and update anomalies.

Integrity rules should include unique passenger emails and phone numbers, unique driver licence numbers, unique vehicle registration numbers, valid ride statuses, non-negative fares and payment amounts, rating scores from 1 to 5, valid latitude/longitude ranges, and timestamp checks. Filtered unique indexes can enforce no more than one successful payment per ride and at most one active assignment per driver or vehicle. These indexes do not by themselves prevent every possible historical time-overlap.

Design note: PickupLocation and Destination are VARCHAR fields in this portfolio version. A production system could introduce a LOCATION table or spatial data types for reusable addresses and geospatial queries.

5. Final note

The relational schema for SwiftRide Nigeria was derived from the Entity–Relationship model and comprises nine tables: PASSENGER, DRIVER, VEHICLE, VEHICLE_ASSIGNMENT, FARE_RULE, RIDE, PAYMENT, RATING and DRIVER_LOCATION. Each table uses a primary key to identify its records. Foreign keys establish relationships between related tables and support referential integrity.

The RIDE table is the central operational relation because it connects passengers, drivers and fare rules. VEHICLE_ASSIGNMENT maintains the history of driver–vehicle assignments, while PAYMENT accommodates multiple payment attempts for a ride. RATING supports optional passenger feedback, and DRIVER_LOCATION stores timestamped geographical coordinates for driver tracking.

The schema incorporates unique constraints and validation rules to improve data quality. Separating passenger, driver, vehicle and fare-rule information into distinct relations reduces unnecessary duplication and helps prevent insertion, update and deletion anomalies. The schema provides a foundation for Microsoft SQL Server implementation and subsequent work on SQL queries, optimisation, transactions and database integrity.